

Arquitectura del Catálogo Interactivo

Proyecto: **Caps-Vnzla** | Stack: Python, Flask, SQLite & SQLAlchemy

1. Dependencias y Librerías

A continuación se detallan las librerías necesarias en requirements.txt para garantizar una arquitectura modular, escalable y segura:

Librería	Versión	Función / Propósito
Flask	3.0.2	Framework web principal ligero.
Flask-SQLAlchemy	3.1.1	ORM para gestión relacional de datos en SQLite.
Flask-Migrate	4.0.7	Control de versiones y migraciones de BDD.
Flask-WTF / WTForms	1.2.1 / 3.1.2	Formularios seguros y validación de entradas.
Pillow	10.2.0	Procesamiento y optimización de imágenes.
python-dotenv	1.0.1	Manejo seguro de variables de entorno.

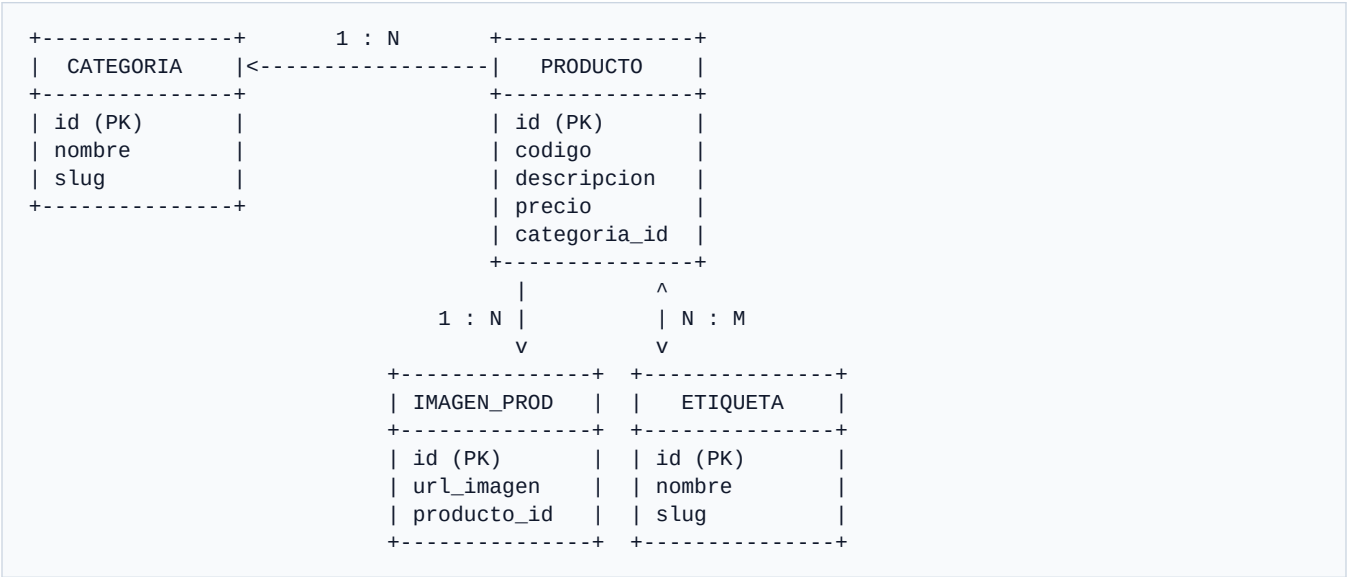
2. Estructura del Proyecto (Application Factory)

Diseño modular optimizado para mantener separación clara de responsabilidades (Patrón MVC):

```
Caps-Vnzla/
├── app/
│   ├── __init__.py      # Inicialización de la App y Flask-SQLAlchemy
│   ├── models.py        # Modelos de BDD en SQLite (SQLAlchemy)
│   ├── static/          # CSS, JavaScript e imágenes subidas
│   │   ├── css/style.css
│   │   ├── js/catalog.js # Lógica interactiva cliente (AJAX/Filtros)
│   │   ├── uploads/      # Galería de imágenes de productos
│   │   └── templates/    # Plantillas Jinja2 (HTML)
│   │       ├── base.html
│   │       ├── catalog/  # Vistas del catálogo público
│   │       └── admin/    # Panel de administración
│   └── views/           # Controladores / Blueprints
│       ├── catalog.py    # Rutas del catálogo y API
│       └── admin.py      # Rutas de gestión
├── config.py            # Configuración de SQLite y variables
├── .env                 # Variables de entorno
├── requirements.txt     # Lista de dependencias
└── run.py               # Punto de entrada de la aplicación
```

3. Modelo Entidad-Relación (Base de Datos SQLite)

Estructura relacional diseñada para soportar filtrado dinámico, categorías y múltiples imágenes por producto:



4. Código Fuente Principal

A. Definición de Modelos (app/models.py)

```

from app import db
from datetime import datetime

producto_etiqueta = db.Table('producto_etiqueta',
    db.Column('producto_id', db.Integer, db.ForeignKey('productos.id'), primary_key=True),
    db.Column('etiqueta_id', db.Integer, db.ForeignKey('etiquetas.id'), primary_key=True)
)

class Categoria(db.Model):
    __tablename__ = 'categorias'
    id = db.Column(db.Integer, primary_key=True)
    nombre = db.Column(db.String(100), nullable=False)
    slug = db.Column(db.String(100), unique=True, nullable=False)
    productos = db.relationship('Producto', backref='categoria', lazy=True)

class Producto(db.Model):
    __tablename__ = 'productos'
    id = db.Column(db.Integer, primary_key=True)
    codigo = db.Column(db.String(50), unique=True, nullable=False)
    nombre = db.Column(db.String(150), nullable=False)
    descripcion = db.Column(db.Text, nullable=False)
    precio = db.Column(db.Float, nullable=False)
    activo = db.Column(db.Boolean, default=True)
    categoria_id = db.Column(db.Integer, db.ForeignKey('categorias.id'), nullable=False)
    imagenes = db.relationship('ImagenProducto', backref='producto', cascade="all, delete-orphan", lazy=True)
    etiquetas = db.relationship('Etiqueta', secondary=producto_etiqueta, lazy='subquery')

class ImagenProducto(db.Model):
    __tablename__ = 'imagenes_producto'
    id = db.Column(db.Integer, primary_key=True)
    url_imagen = db.Column(db.String(255), nullable=False)
    es_principal = db.Column(db.Boolean, default=False)
    producto_id = db.Column(db.Integer, db.ForeignKey('productos.id'), nullable=False)

```

B. Controlador e Interacción AJAX (app/views/catalog.py)

```

from flask import Blueprint, render_template, request, jsonify
from app.models import Producto, Categoria

catalog_bp = Blueprint('catalog', __name__)

@catalog_bp.route('/')
def index():
    categorias = Categoria.query.all()
    return render_template('catalog/index.html', categorias=categorias)

@catalog_bp.route('/api/productos')
def get_productos():
    categoria_id = request.args.get('categoria_id', type=int)
    busqueda = request.args.get('q', type=str)

    query = Producto.query.filter_by(activo=True)
    if categoria_id:
        query = query.filter_by(categoria_id=categoria_id)
    if busqueda:
        query = query.filter(Producto.nombre.ilike(f'%{busqueda}%'))

    productos = query.all()
    resultado = [{
        'id': p.id,
        'nombre': p.nombre,
        'precio': p.precio,
        'categoria': p.categoria.nombre,
        'imagen': next((i.url_imagen for i in p.imagenes if i.es_principal), '/static/img/placeholder.png')
    } for p in productos]

    return jsonify(resultado)

```

5. Pasos para Inicializar el Proyecto

1. **Crear entorno virtual:** `python -m venv venv`
2. **Activar entorno:** `source venv/bin/activate` (Linux/Mac) o `venv\Scripts\activate` (Windows)
3. **Instalar librerías:** `pip install -r requirements.txt`
4. **Ejecutar servidor local:** `python run.py`